| Filière : | Développement des Systèmes d'Information (DSI) | Durée : | 3 heures |
|---|---|---|---|
| Épreuve : | Développement des Applications Informatiques – DAI - | Coefficient : | |

## CONSIGNES

- ✓ **Le sujet comporte 2 dossiers ;**
- ✓ **chaque dossier doit être traité dans une feuille séparée.**

**Barème de notation**

| | |
|---|---|
| **Dossier 1** : Gestion des branchements électriques. | 14 points |
| **Dossier 2** :Consultation des demandes de branchement. | 08 points |
| **Total** | **22 points** |

- ✓ **Il sera pris en considération la qualité de la rédaction lors de la correction.**
- ✓ **Aucun document n'est autorisé.**

# ÉTUDE DE CAS : BRANCHEMENT DE L'EAU POTABLE ET D'ÉLECTRICITÉ

La société **MSEPE** ( ***M****ulti-Service pour le branchement de l'****E****au* ***P****otable et d'****É****lectricité* ) offre des divers services parmi eux les branchements d'électricité et de l'eau potable au Maroc. La société sous-traite une partie de ses activités en délégant à des entreprises tierces la réalisation des branchements chez les clients. Dans cette étude on s'intéresse seulement aux branchements d'électricité.

Le traitement d'un branchement se déroule en plusieurs étapes :

- ✓ L'enregistrement de la demande de branchement d'un client et la validation de ses informations,
- ✓ L'élaboration du devis correspondant à la demande,
- ✓ La gestion des plannings, la communication des dates et lieux des rendez-vous aux sous-traitants,
- ✓ La réalisation des branchements par les sous-traitants,
- ✓ L'enquête de qualité afin de mesurer le degré de satisfaction des clients ainsi que la qualité du travail réalisé par les sous-traitants et par la société **MSEPE**.

Pour organiser les branchements, les villes sont découpées en **ZEI** (***Z****ones* ***É****lémentaires d'****I****ntervention*). Une **ZEI** correspond à un secteur autour d'une **commune**.

## DOSSIER 1 : GESTION DES BRANCHEMENTS ÉLECTRIQUES *(14 pts)*

Le diagramme de classes suivant présente une partie du système de gestion des branchements électriques (*figure 1* ) :

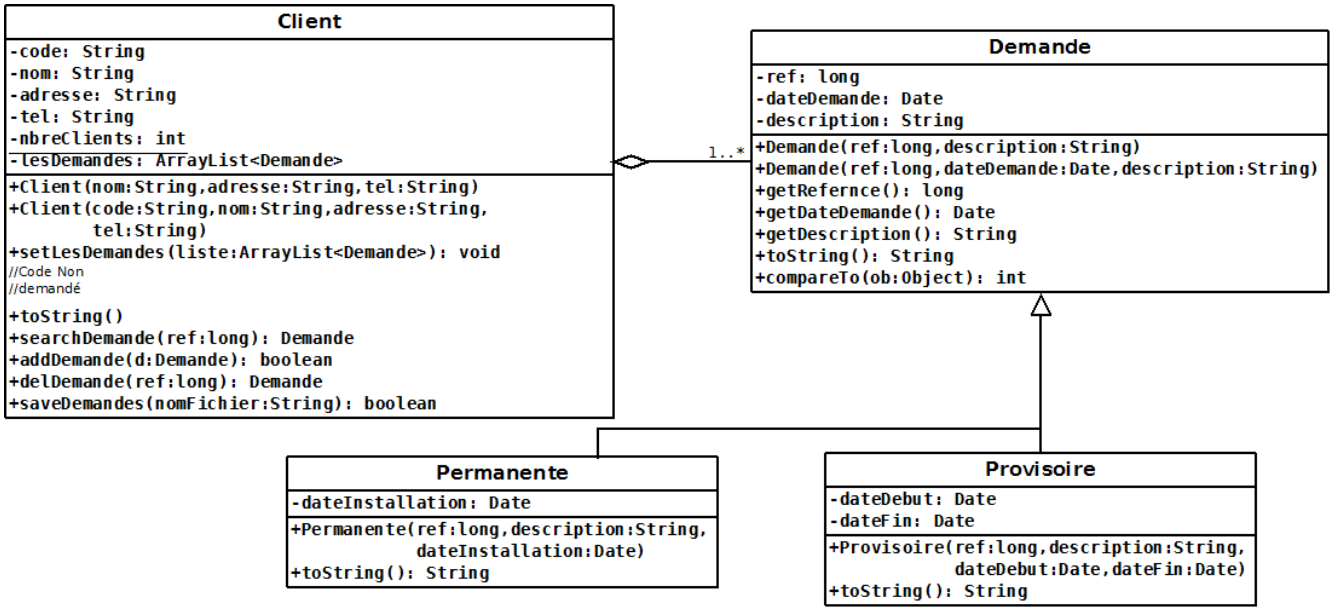


**Figure 1 :** Diagramme de classes **"Gestion des demandes de branchement"**

1. Créer la classe **« Demande »** qui implémente les deux interfaces « **Serializable** », « **Comparable** » ***et*** qui contient : *(0,5 pt)*
   - Deux constructeurs, le premier avec deux paramètres (*référence et description*), la date de la demande sera celle du système et le deuxième avec trois paramètres qui permet d'initialiser tous les attributs. *(1 pt)*
   - La méthode « **toString** » retourne une chaîne porteuse d'informations sur une demande.
     La chaîne aura la forme suivante : *(0,5 pt)*
     **Référence : xxxx, Description : xxxx, Date de la demande : jj/mm/aaaa.**
   - Les accesseurs pour tous les attributs de la classe. *(0,5 pt)*
   - La méthode « **compareTo** » compare deux demandes par leurs **références**. *(1 pt)*
2. Écrire le code de la classe d'exception « **ErreurDate** » qui permet de récupérer un message d'erreur. *(1 pt)*
3. Implémenter la classe **« Permanente»** qui contient : *(0,5 pt)*
   - Un constructeur, avec trois paramètres, qui génère l'exception « **ErreurDate** » si la date de la demande (*date du système*) est supérieure à la date d'installation. Le message d'erreur est : "*Erreur de date* ". *(1 pt)*
   - La méthode « **toString** » retourne une chaîne sous la forme suivante : *(1 pt)*
     **Référence : xxxx, Description : xxxx, Date de la demande : jj/mm/aaaa, Date d'installation : jj/mm/aaaa.**

   **NB :** Le code de la classe Provisoire est déjà implémenté.
4. Implémenter la classe **« Client »** qui contient : *(0,5 pt)*
   - Un constructeur avec trois paramètres permettant d'initialiser : *(1 pt)*
     - Le nom, l'adresse et le téléphone.
     - L'attribut de classe « **nbreClients** » compte le nombre de clients instanciés.
     - Le « **Code** » par le nom du client suivi du numéro de l'ordre de création de ce client.
     - La collection « **lesDemandes** » de type « **ArrayList** ».

     **NB :** on suppose que le constructeur de quatre paramètres est déjà implémenté.
   - La méthode « **toString** » permet de retourner une chaîne sous la forme suivante : *(1 pt)*

**Code:xxxx, Nom : xxxx, Adresse : xxxx, Tel : xxxx**
**Les demandes :**
**Référence : xxxx, Date de la demande : jj/mm/aaaa, Date d'installation : jj/mm/aaaa**
…… …… …… …… ……
**Référence : xxxx, Date de la demande : jj/mm/aaaa, Date de début : jj/mm/aaaa, Date de fin : jj/mm/aaaa**
…… …… …… …… ……

- La méthode « **addDemande(Demande d)** » qui ajoute une nouvelle demande, donnée en paramètre, à la collection et retourne l'état de l'opération. *(1 pt)*
- La méthode « **searchDemande(long ref)** » qui recherche et retourne une demande en se basant sur sa référence. Si cette demande n'existe pas la méthode retourne « **null** ». *(1 pt)*
- La méthode « **delDemande(long ref)** » qui supprime une demande de la collection en se basant sur sa référence et retourne la demande supprimée. *(1 pt)*
- La méthode « **saveDemandes(String nomFichier)** » qui sauvegarde, dans un fichier d'objets, les demandes permanentes de ce client. *(1,5 pt)*

## DOSSIER2 : CONSULTATION DES DEMANDES DE BRANCHEMENT *(8 pts)*

La mise en œuvre d'une nouvelle application est envisagée pour permettre aux sous-traitants de consulter les demandes de branchement des clients et d'enregistrer leurs indisponibilités. Cette application sera hébergée dans un serveur *d'application* qui exploite les objets distribués via l'utilisation de la **RMI** (***RemoteMethod Invocation***).

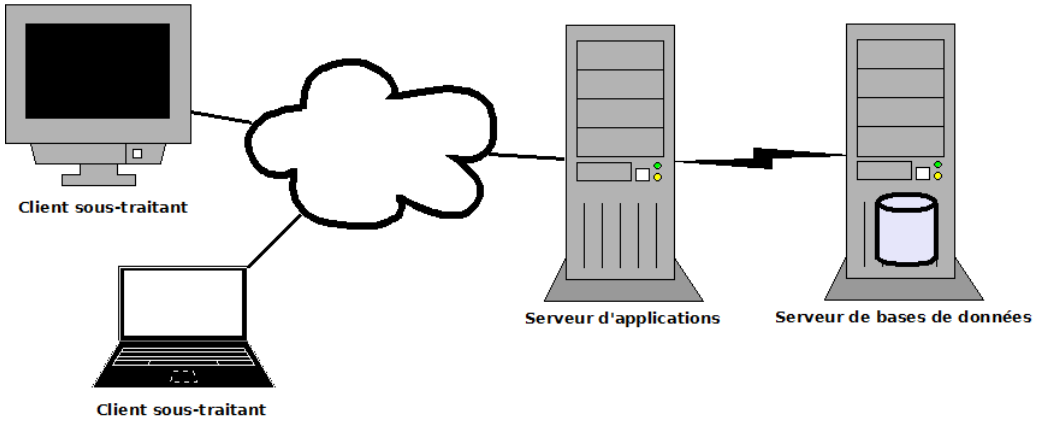


**Figure 2 :** Architecture client/serveur

Le serveur d'application utilise l'API RMI intégrant une classe représentant des méthodes à partager. Ces méthodes permettent d'accéder à la base de données **« BD_Branchement »** implantée dans un autre serveur de Gestion de Base de Données. Les postes des sous-traitants (clients) exploitent les méthodes offertes par le serveur d'application (*figure* 2).

1. Préciser le type d'architecture client/serveur adopté par ce système et donner le modèle **GartnerGroup** correspondant. *(1 pt)*

Voici un extrait du modèle relationnel de la base de données **"BD_Branchement"** (*figure* 3 ):

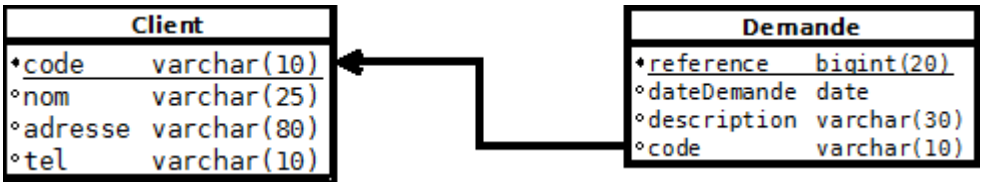


**Figure 3 :** MLD **"BD_Branchement"**

Soit la classe suivante qui représente l'objet distant :

```
public class ServeurDistant extends Unicast RemoteObject implements IntServiceBranchement{
      private ArrayList<Client> liste;
      private Connection cnx;

      public ServeurDistant() throws RemoteException {
            . . . . . . . . . . . . . // Code à compléter.
      }

      public ArrayList<Demande> lireDemandes(String code) throws RemoteException {
            . . . . . . . . . . . . // Code à compléter.
      }

      public void lireClients() throws RemoteException {
            . . . . . . . . . . . . // Code à compléter.
      }
}
```

2. Donner le code de l'interface « **IntServiceBranchement**» implémentée par la classe « **ServeurDistant** » afin d'assurer le partage des méthodes de cette classe. *(1 pt)*

3. Compléter la classe « **ServeurDistant** » par l'implémentation des méthodes suivantes :
   - Un constructeur permettant d'instancier l'attribut « **liste** » des clients et d'établir une connexion à la base de données MYSQL. *(1 pt)*
     - ✓ **URL**:jdbc:mysql://Administrateur:3306/bd_branchement
     - ✓ **User** :*root*
     - ✓ ***Password*** :DSI2025.

     En cas d'échec de connexion, afficher un message d'erreur.
   - La méthode « **lireDemandes** » permet de retourner une collection des demandes dont le code client est donné en paramètre. *(1 pt)*
   - La méthode « **lireClients** » permet de stocker tous les clients de la table « **Client** » de la base de données dans l'attribut « **liste** ». Pour chaque client de cette collection, on doit récupérer ses demandes en les sauvegardant dans l'attribut « **lesDemandes** » de l'objet client. *(1 pt)*

4. Écrire le code du ***serveur RMI*** permettant de démarrer l'annuaire ***rmiregistry***, de créer l'objet partagé de la classe « **ServeurDistant** » et de publier la référence de cet objet dans l'annuaire sous le nom «**ob**». *(1,5 pt)*

5. Écrire le code du ***client RMI*** permettant de : *(1,5 pt)*
   - Récupérer, de l'annuaire, la référence de l'objet distant,
   - Lire et afficher la liste des demandes d'un client dont le code est : « alkendi 2025».